

UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA



Curso:

Base de Datos II

Laboratorio 04: Sequential File vs AVL File

Alumno:

Llerena Silva, Nicolás Alejandro (202110190)

Profesor:

Lovon, Percy

Lima, setiembre de 2026

Introducción

El laboratorio pide implementar dos organizaciones de archivo sobre memoria secundaria y compararlas usando el `Employee_ID` como clave: un archivo secuencial ordenado con espacio auxiliar y un archivo organizado como árbol AVL con los punteros guardados en el propio archivo. Sobre cada uno se implementaron `insert`, `search`, `remove` y `rangeSearch`, y después se midió cuánto cuesta cada operación al crecer el archivo.

Todo está escrito en Python con la librería estándar; solo el script de gráficas y el que arma este documento usan `matplotlib` y `python-docx`. Los archivos son binarios de longitud fija y las mediciones se hicieron en una MacBook con SSD, sobre el mismo conjunto de datos para las dos estructuras.

Los datos y el registro

El enunciado parte de un `employee.csv` que no venía con el material, así que lo construí desde la base `employees` del Laboratorio 01, que ya tiene empleados reales con nombre, departamento, puesto, sueldo y fecha de ingreso. El script `exportar_csv.py` toma los 20 500 empleados vigentes (los que tienen departamento, título y salario con `to_date = 9999-01-01`) y arma el CSV con las ocho columnas que pide el enunciado. El único campo que la base no tiene es `Country`, así que se asigna de forma determinista a partir del `id`; los otros siete son datos reales. Si más adelante aparece el `employee.csv` oficial, los dos programas lo cargan igual: el lector normaliza los nombres de columna y no depende del orden en que vengan.

El registro sigue exactamente la tabla del enunciado y ocupa 112 bytes. Las cadenas se guardan rellenas con espacios hasta su tamaño máximo y se recortan al leer. Vale la pena anotar un detalle del `Salary`: al pedirse en 4 bytes es un float de precisión simple, que tiene unos siete dígitos significativos, así que sueldos como 95000 se guardan exactos pero un valor con muchos decimales se redondea al leerlo.

Campo	Tipo	Bytes	Desplazamiento
<code>Employee_ID</code>	<code>int</code>	4	0
<code>Employee_Name</code>	<code>string</code>	30	4
<code>Age</code>	<code>int</code>	4	34
<code>Country</code>	<code>string</code>	20	38

Department	string	20	58
Position	string	20	78
Salary	float	4	98
Joining_Date	string DD/MM/YYYY	10	102
Total		112	

```

bash - labs/4

$ python3 exportar_csv.py 20500
data/employee.csv: 20500 registros, 1508753 bytes
Employee_ID,Employee_Name,Age,Country,Department,Position,Salary,Joining_Date
10001,Georgi Facello,73,Brasil,Development,Senior Engineer,95000,26/06/1986
10002,Bezalel Simmel,62,Espana,Sales,Staff,88000,21/11/1985
10003,Parto Bamford,66,Uruguay,Production,Senior Engineer,43311,28/08/1986
$

```

Exportación del CSV desde la base employees del Laboratorio 01.

P1: archivo secuencial

El archivo secuencial son dos archivos: el de datos, ordenado por Employee_ID, y uno auxiliar donde caen las inserciones sin orden. Cada registro se guarda con un byte extra de estado al final, así que ocupa 113 bytes en disco: 112 de datos y uno que vale 1 si el registro está activo y 0 si fue eliminado. No hay cabecera porque la cantidad de registros sale del tamaño del archivo dividido entre 113.

insert escribe siempre al final del auxiliar, que cuesta un solo acceso. Cuando el auxiliar supera k registros se dispara la reconstrucción: se lee el archivo de datos de principio a fin, se mezcla con el auxiliar ya ordenado en memoria (nunca son más de k registros) y se escribe un archivo nuevo que reemplaza al anterior. En esa mezcla se descartan los registros marcados como eliminados, que es donde el borrado lógico se hace efectivo. La mezcla es en streaming, o sea que el archivo de datos nunca se carga entero en memoria.

search hace búsqueda binaria sobre el archivo de datos, que son $\log_2(n)$ accesos, y si no encuentra la clave recorre el auxiliar, que son a lo más k accesos más. remove usa la misma búsqueda y se limita a poner el byte de estado en 0, sin mover nada de sitio. rangeSearch busca con búsqueda binaria la primera clave mayor o igual al inicio del rango y desde ahí avanza secuencialmente mientras la clave siga dentro del rango, que es la parte donde

este diseño se luce: los registros que necesita están uno al lado del otro en el archivo.

```
bash - labs/4

$ python3 sequential_file.py
Sequential File sobre data/employee.csv
  tamaño de registro : 113 bytes (112 de datos + 1 de estado)
  registros cargados : 2000
  k del auxiliar      : 16
  search(11247) -> Ranga Rebaine, Human Resources [10 accesos]
  insert(999999) -> auxiliar con 1 registros [1 accesos]
  search(999999) -> Nicolas Llerena
  tras superar k      : reconstrucciones=1, auxiliar=0, datos=2017
  remove(11247) -> True [9 accesos]
  search(11247) tras eliminar -> None
  rangeSearch(10013, 10513) -> 402 empleados [414 accesos]
    10013 Eberhardt Terkki      Human Resources
    10014 Berni Genin          Development
    10016 Kazuhito Cappelletti Sales
  registros validos   : 2016
$
```

Prueba de las cuatro operaciones sobre 2000 empleados, con $k = 16$. Entre corchetes van los accesos a disco de cada operación.

P2: archivo AVL

En el archivo AVL cada nodo es un registro de 112 bytes seguido de tres enteros: la posición del hijo izquierdo, la del derecho y la altura del subárbol. Son 124 bytes por nodo y las posiciones son índices dentro del archivo, no offsets en bytes, con -1 como puntero nulo. Al inicio del archivo hay una cabecera de 12 bytes con la posición de la raíz, la cabeza de la lista de posiciones libres y la cantidad de nodos vivos.

insert baja por el árbol comparando el Employee_ID, crea el nodo en la primera posición libre que encuentre y al volver de la recursión recalcula alturas y rebalancea. Las rotaciones son las cuatro clásicas y cada una reescribe los dos nodos que cambian de lugar más los punteros del padre, todo sobre el archivo. remove es la eliminación estándar del AVL: si el nodo tiene un hijo o ninguno se enlaza el hijo al padre y la posición liberada se encadena en la lista de libres para reutilizarla en la siguiente inserción; si tiene dos hijos se copia el registro

del sucesor inorden y se elimina ese sucesor del subárbol derecho. En ambos casos se rebalancea al regresar.

search es un descenso iterativo, un acceso por nivel. rangeSearch es un recorrido inorden podado: si la clave del nodo ya es menor o igual al inicio del rango no se baja por la izquierda, y si es mayor o igual al final no se baja por la derecha, así que solo se visitan los nodos del rango más el camino que lleva a ellos. Como el recorrido inorden de un árbol de búsqueda sale ordenado, el resultado ya viene ordenado por Employee_ID sin necesidad de ordenarlo después.

```
bash - labs/4

$ python3 avl_file.py
AVL File sobre data/employee.csv
  tamaño de nodo      : 124 bytes (112 de datos + 12 de enlaces)
  nodos en el árbol   : 2000
  altura del árbol    : 11 (log2(2000) = 10)
  search(11247) -> Ranga Rebaine, Human Resources [11 accesos]
  insert(999999) -> altura=11, nodos=2001 [50 accesos]
  search(999999) -> Nicolas Llerena
  remove(11247) -> True [41 accesos], altura=11, nodos=2000
  search(11247) tras eliminar -> None
  rangeSearch(10013, 10513) -> 402 empleados [408 accesos]
      10013  Eberhardt Terkki           Human Resources
      10014  Berni Genin                Development
      10016  Kazuhito Cappelletti      Sales
  recorrido inorden   : 2000 registros, ordenado=True
$
```

Las mismas pruebas sobre el AVL. La altura 11 con 2000 nodos confirma que el árbol quedó balanceado, y el recorrido inorden devuelve los registros en orden.

P3: evaluación de desempeño

Para medir usé cinco tamaños de archivo (1000, 2500, 5000, 10 000 y 20 000 registros) tomados del mismo CSV barajado con una semilla fija, y 500 registros apartados que nunca entran en la construcción y sirven para las pruebas de inserción. Sobre cada tamaño se ejecutan 500 búsquedas de claves existentes, 100 búsquedas por rango de 250 unidades de ancho, las 500 inserciones y 500 eliminaciones, siempre con las mismas claves para las dos

estructuras. Además de cronometrar cada operación cuento los accesos a disco, que son las lecturas y escrituras de registro o de nodo que hace cada estructura; esa métrica es la que de verdad describe el costo en memoria secundaria, porque el tiempo se contamina con la caché del sistema operativo.

El benchmark también compara los resultados de las dos estructuras: cada búsqueda y cada rango tienen que devolver lo mismo en el archivo secuencial y en el AVL, y al final los dos deben contener el mismo conjunto de registros válidos. Esas comprobaciones son asserts dentro del programa, así que si algo no coincidiera el benchmark se caería en vez de reportar números.

Registros	Inserción de 500 (s)		Búsqueda (ms)		Rango (ms)		Eliminación (ms)	
	Sec.	AVL	Sec.	AVL	Sec.	AVL	Sec.	AVL
1000	0,05	0,05	0,025	0,018	0,052	0,044	0,044	0,090
2500	0,09	0,05	0,027	0,021	0,069	0,093	0,042	0,097
5000	0,15	0,06	0,029	0,023	0,095	0,139	0,046	0,105
10000	0,28	0,06	0,030	0,026	0,149	0,257	0,049	0,118
20000	0,54	0,07	0,033	0,030	0,283	0,541	0,051	0,129

Tiempos por operación. La inserción es el total de las 500; las demás son el promedio por operación.

Registros	Altura AVL	Inserción		Búsqueda		Rango		Eliminación	
		Sec.	AVL	Sec.	AVL	Sec.	AVL	Sec.	AVL
1000	12	77	47	8.9	9.2	22	20	11	43
2500	14	167	51	10.4	10.6	38	36	12	47
5000	15	317	55	11.5	11.5	63	61	12	51
10000	16	617	59	12.4	12.7	113	111	13	55
20000	17	1217	63	13.4	13.7	212	210	14	59

Accesos a disco promedio por operación.

```

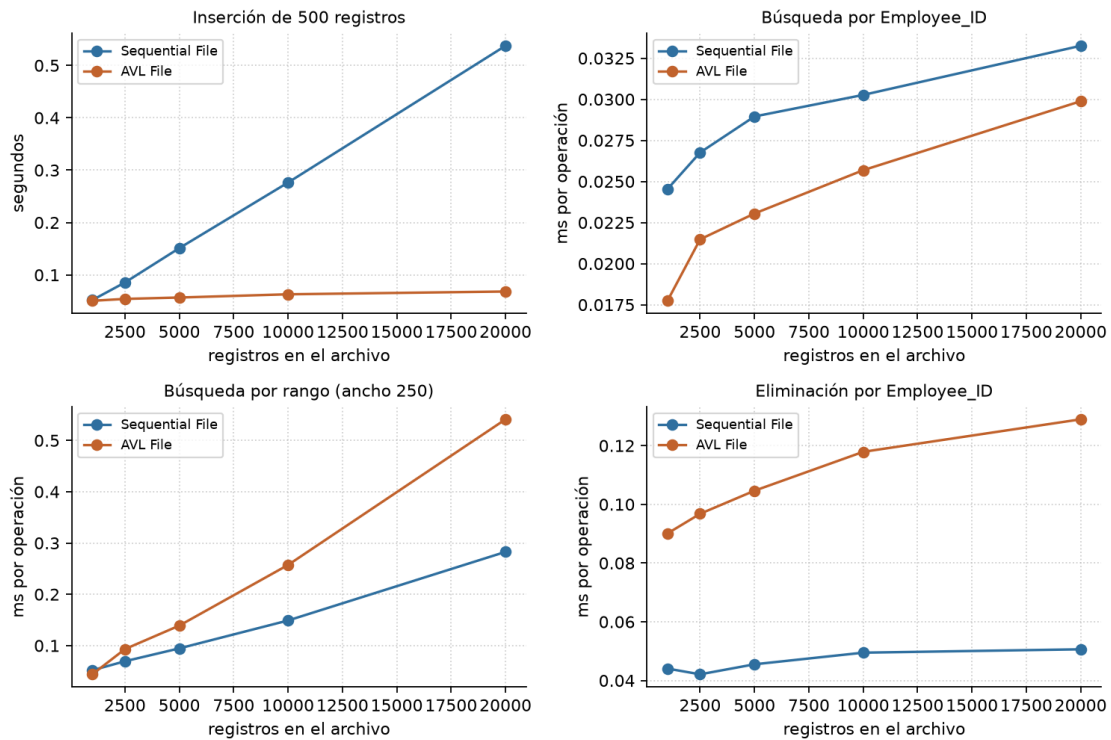
bash - labs/4

$ python3 benchmark.py
n= 1000 altura_avl=12 reconstrucciones= 15 busqueda seq=0.025 ms / avl=0.018 ms | insercion seq= 0.05 s / avl=0.05 s
n= 2500 altura_avl=14 reconstrucciones= 15 busqueda seq=0.027 ms / avl=0.021 ms | insercion seq= 0.09 s / avl=0.05 s
n= 5000 altura_avl=15 reconstrucciones= 15 busqueda seq=0.029 ms / avl=0.023 ms | insercion seq= 0.15 s / avl=0.06 s
n= 10000 altura_avl=16 reconstrucciones= 15 busqueda seq=0.030 ms / avl=0.026 ms | insercion seq= 0.28 s / avl=0.06 s
n= 20000 altura_avl=17 reconstrucciones= 15 busqueda seq=0.033 ms / avl=0.030 ms | insercion seq= 0.54 s / avl=0.07 s
k= 8 reconstrucciones= 55 auxiliar= 8 insercion= 1.01 s busqueda en datos= 12.5 accesos en auxiliar= 20.7 accesos
k= 16 reconstrucciones= 29 auxiliar= 16 insercion= 0.52 s busqueda en datos= 12.6 accesos en auxiliar= 25.7 accesos
k= 32 reconstrucciones= 15 auxiliar= 32 insercion= 0.27 s busqueda en datos= 12.5 accesos en auxiliar= 32.5 accesos
k= 64 reconstrucciones= 7 auxiliar= 64 insercion= 0.14 s busqueda en datos= 12.5 accesos en auxiliar= 68.6 accesos
k= 128 reconstrucciones= 3 auxiliar= 128 insercion= 0.07 s busqueda en datos= 12.4 accesos en auxiliar= 134.3 accesos

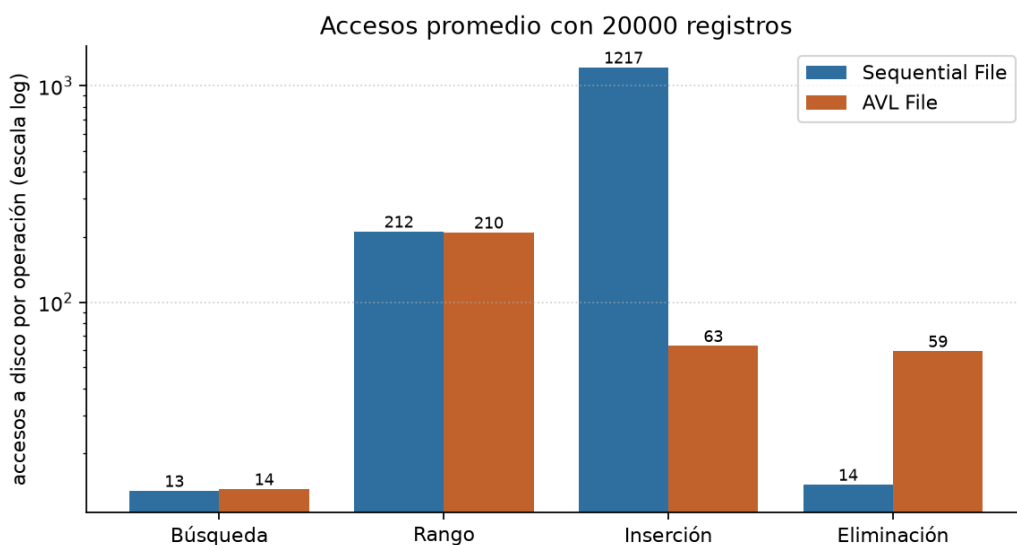
resultados en resultados/
$

```

Salida del benchmark completo.



Tiempos de las cuatro operaciones al crecer el archivo.



Accesos a disco por operación con 20 000 registros, en escala logarítmica.

Análisis por operación

Insertión. Es la diferencia más grande y la que decide la comparación. El AVL se mantiene plano: pasa de 47 a 63 accesos por inserción entre 1000 y 20 000 registros, un crecimiento logarítmico, y las 500 inserciones tardan 0,07 s en el archivo más grande. El secuencial va de 77 a 1217 accesos, o sea que crece de forma lineal con el tamaño del archivo, y termina tardando 0,54 s, 7,9 veces

más. La inserción en sí es barata, un acceso al auxiliar; lo caro son las 15 reconstrucciones que se disparan, cada una leyendo y reescribiendo el archivo completo.

Búsqueda. Aquí quedan parejos, y tiene sentido: los dos son logarítmicos. Con 20 000 registros la búsqueda binaria usa 13.4 accesos y el AVL 13.7, con el árbol apenas por encima porque su altura (17) es mayor que $\log_2(20\ 000) \approx 14,3$; un AVL puede llegar a 1,44 veces esa cota. En tiempo el AVL sale levemente mejor (0,030 ms contra 0,033 ms) porque la búsqueda binaria salta a posiciones muy separadas del archivo mientras que los nodos altos del árbol se visitan en todas las búsquedas y quedan en caché. La diferencia es de microsegundos: para búsquedas puntuales las dos estructuras son equivalentes.

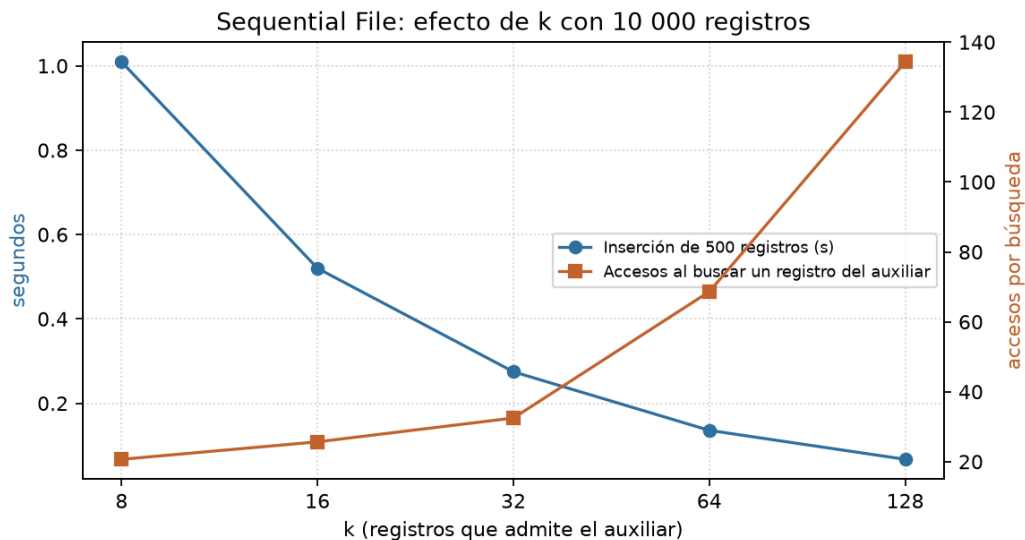
Búsqueda por rango. Se invierte la ventaja. Los accesos son casi idénticos (212 contra 210 con 20 000 registros, porque los dos terminan tocando los ~197 registros del rango), pero el tiempo del secuencial es la mitad: 0,283 ms contra 0,541 ms. La razón es la localidad: el secuencial encuentra el inicio del rango y de ahí lee registros contiguos, que el sistema operativo trae de a bloques enteros, mientras que el AVL recorre el árbol saltando entre nodos dispersos por todo el archivo. Es exactamente el argumento por el que un índice clustered le gana a uno no clustered en consultas de rango.

Eliminación. Gana el secuencial y con holgura: 14 accesos contra 59 del AVL. Pero la comparación es un poco tramposa, porque no hacen lo mismo. El secuencial hace borrado lógico: busca el registro y le cambia un byte, así que cuesta lo mismo que una búsqueda más una escritura, y el espacio sigue ocupado hasta la próxima reconstrucción. El AVL elimina de verdad: reestructura el árbol, puede rotar en todo el camino de vuelta a la raíz y devuelve la posición liberada a la lista de libres para reutilizarla. El secuencial paga después lo que no paga ahora.

El parámetro k del archivo secuencial

El tamaño del auxiliar es la perilla que ajusta el archivo secuencial, así que medí aparte cómo se comporta con 10 000 registros. Con $k = 8$ las 500 inserciones provocan 55 reconstrucciones y tardan 1,01 s; con $k = 128$ bajan a

3 reconstrucciones y 0,07 s, catorce veces más rápido. El precio aparece del otro lado: buscar un registro que todavía está en el auxiliar cuesta 21 accesos con $k = 8$ y 134 con $k = 128$, porque el auxiliar se recorre de forma lineal. Las búsquedas que caen en el archivo de datos no se enteran: se mantienen en unos 13 accesos en todos los casos.



Efecto de k sobre el costo de insertar y sobre el de buscar en el auxiliar.

Cuándo conviene cada uno

El archivo secuencial conviene cuando los datos casi no cambian y lo que se hace sobre ellos son lecturas, sobre todo por rango: reportes, históricos, tablas de referencia, cualquier cosa que se cargue una vez y después solo se consulte. Su construcción es prácticamente gratis comparada con la del AVL (0,02 s contra 2,4 s para 20 000 registros, porque una es ordenar y volcar y la otra son 20 000 inserciones con rotaciones), la búsqueda binaria es óptima en accesos y el barrido por rango aprovecha la localidad del disco. También es más simple de implementar y de recuperar ante una falla.

El AVL conviene cuando el archivo se modifica seguido y no se puede pagar una reconstrucción cada tantas inserciones: catálogos que crecen todo el día, tablas transaccionales, cualquier caso donde insertar y borrar sean tan frecuentes como consultar. Mantiene el costo logarítmico sin importar el orden en que lleguen las claves y su eliminación libera espacio de inmediato. A cambio, cada nodo carga 12 bytes de punteros (un 11 % de sobrecarga sobre el registro) y el recorrido por rango pierde localidad.

Dicho de otra forma: si el archivo se lee mucho más de lo que se escribe, gana el secuencial; si se escribe seguido, gana el AVL. Y si hicieran falta las dos cosas a la vez, el camino no sería elegir uno sino usar un árbol B+, que junta la altura logarítmica del AVL con las hojas enlazadas que dan localidad en los rangos.

Conclusiones

Las dos estructuras resuelven las cuatro operaciones sobre los mismos datos y devuelven exactamente los mismos resultados, así que la comparación es sobre el costo y no sobre la funcionalidad. La búsqueda puntual es un empate, porque las dos son logarítmicas. En inserción el AVL gana claramente y la distancia crece con el archivo, ya que el secuencial reconstruye en tiempo lineal. En rango gana el secuencial por localidad, con la mitad del tiempo pese a hacer los mismos accesos. Y en eliminación gana el secuencial solo porque no hace el trabajo completo: marca el registro y difiere el costo real a la siguiente reconstrucción.

La medición también deja ver que contar accesos a disco y cronometrar no son lo mismo. En el rango los accesos son iguales y los tiempos difieren al doble, y en la búsqueda pasa al revés. Cuando el archivo entra en la caché del sistema operativo, como acá, lo que termina pesando es el patrón de acceso más que la cantidad.

Anexo: archivos entregados

registro.py define el registro de 112 bytes, su empaquetado con struct y la lectura del CSV. sequential_file.py implementa la clase SequentialFile con build, insert, search, remove, range_search y la reconstrucción por streaming. avl_file.py implementa AVLFile con las mismas operaciones sobre el árbol en disco, incluyendo rotaciones y lista de posiciones libres. exportar_csv.py arma data/employee.csv desde PostgreSQL y benchmark.py corre las mediciones y genera las gráficas. Los tres módulos principales se ejecutan solos y vuelven a imprimir las salidas que aparecen en las capturas de este informe.